

JavaScript Fundamentals Assignment

Assignment 1 — Variables, Conditionals, Types, Coercion & Functions

Student Name: _____ Date: _____ Score: _____ / 105

Instructions: Complete all questions. Write or type your JavaScript solution in the spaces provided. You can test your code in the browser DevTools Console or Node.js before finalizing your answer. Total Marks: **105** (100 core + 5 bonus).

SECTION A: VARIABLES & DATA TYPES

10 MARKS

Q1. Variable Declarations & Type Inspection

5 Marks

Create the following four variables using the **correct** keyword (`const` or `let`):

- Your full name (it won't change)
- Your current age (it changes every birthday)
- Your school name (it won't change)
- Whether you are a student (`true` / `false`)

Then `console.log()` each one alongside its `typeof` . Expected format: `"Wasiu - string"`

// Write your JavaScript code here:

Q2. Predict the Output (Data Types)

5 Marks

Write your predicted output for each expression **before** testing in the console:

Expression	Your Prediction	Actual Console Output
<code>typeof null</code>		
<code>typeof undefined</code>		
<code>typeof 42</code>		
<code>typeof "42"</code>		
<code>typeof true</code>		

SECTION B: TYPE COERCION TRAPS

10 MARKS

Q3. Type Coercion & Strict Equality

10 Marks

Fill in your prediction, verify in the console, and briefly explain **why** JavaScript gave that result:

Code Snippet	Prediction	Actual Result	Why did this happen?
"10" + 5			
"10" - 5			
"3" * "4"			
true + 1			
false + "2"			
"5" === 5			
"5" = 5			

SECTION C: CONDITIONALS & TERNARY OPERATOR

15 MARKS

Q4. Student Grade Classifier (if / else if / else)

6 Marks

Write an `if / else if / else` structure that evaluates a variable `score` and logs the grade:

- **90 – 100:** "A – Excellent"
- **75 – 89:** "B – Very Good"
- **60 – 74:** "C – Good"
- **50 – 59:** "D – Pass"
- **Below 50:** "F – Fail"

// Write your code below and test with at least 3 scores:

Q5. Convert if/else to a One-Liner Ternary

4 Marks

Rewrite this standard if-else block into a clean, single-line ternary statement:

```
let weather;  
if (temperature > 30) {  
  weather = "Hot 🌞";  
} else {  
  weather = "Cool 🧊";  
}
```

// Write your one-line ternary below:

Q6. Logical AND (&&) Graduation Validator

5 Marks

A student can graduate ONLY IF both conditions are met:

1) `age ≥ 18` AND 2) `score ≥ 50`.

Write logic using `&&` and test with:

- a) `age: 20, score: 75` → should graduate
- b) `age: 17, score: 80` → should NOT graduate (too young)
- c) `age: 22, score: 40` → should NOT graduate (score too low)

// Write your code below:

SECTION D: NULLISH COALESCING & OPTIONAL CHAINING

10 MARKS

Q7. Safe Property Access with Optional Chaining (?.)

5 Marks

Given the object below, access each property safely without causing runtime crashes:

```
const student = {  
  name: "Ameen",  
  age: 22,  
  contact: {  
    email: "ameen@school.com",  
    phone: null  
  }  
};
```

- a) Student's name
- b) Student's email (nested inside contact)
- c) Student's phone
- d) Student's address (does not exist)
- e) Student's address city (does not exist — must NOT crash!)

// Write your console.log statements with ?. below:

Q8. Fallback Defaults with Nullish Coalescing (??)

5 Marks

Using the `??` operator, provide fallback values for the student object above:

- If `student.contact?.phone` is null/undefined → default to `"No phone on file"`
- If `student.nickname` is null/undefined → default to `student.name`
- If `student.gpa` is null/undefined → default to `0.0`

// Write your code below:

SECTION E: FUNCTIONS — THE MAIN EVENT

30 MARKS

Q9. Function: `square(number)` — Write 3 Ways

6 Marks

Write a function that takes a number and returns its square (e.g. `square(4)` → `16`). Implement all 3 forms:

```
// 1. Function Declaration: // 2. Function Expression: // 3. Arrow Function: // Test calls:
```

Q10. Function: `isEven(number)` — Write 3 Ways

6 Marks

Write a function returning `true` if a number is even, and `false` if odd. Use modulo (`%`). Implement all 3 forms:

```
// 1. Function Declaration: // 2. Function Expression: // 3. Arrow Function (try the concise one-liner syntax!): // Test calls:
```

Q11. Function: `greetStudent(name, course)` with Default Parameter

8 Marks

Takes `name` and `course`. If `course` is omitted, default it to `"Undeclared"`.

Returns: `"Welcome, [name]! You are studying [course]."`. Implement in all 3 styles:

```
// 1. Function Declaration: // 2. Function Expression: // 3. Arrow Function: // Test calls (test with 2 args AND with 1 arg):
```

Q12. Function: calculateGrade(score) — Arrow Function**5 Marks**

Write an **arrow function** that takes a score and returns the grade string from Q4. Then call it with scores: 95, 82, 67, 55, 33 and log each output.

```
// Write your arrow function and test calls below:
```

Q13. Function: getMaxOfThree(a, b, c) — Declaration**5 Marks**

Write a **function declaration** that takes three numbers and returns the largest. *Rule: Do NOT use*

`Math.max()` — use conditional logic! Example: `getMaxOfThree(10, 25, 7)` → 25

```
// Write your function declaration and test calls below:
```

SECTION F: COMBINE EVERYTHING — CAPSTONE CHALLENGE

25 MARKS

Q14. Student Report Generator (Boss Fight!)

20 Marks

Write a function `generateReport(student)` that accepts an object: `{ name, age, score, course }`

Requirements:

1. Use **Optional Chaining** (`?.`) to safely read all student properties.
2. Use **Nullish Coalescing** (`??`): default course to `"General Studies"`, score to `0`, age to `"N/A"`.
3. Call your `calculateGrade(score)` function from Q12 to get the grade.
4. Use a **Ternary Operator** to compute status: `score ≥ 50 ? "PASSED" : "FAILED"`.
5. Return formatted string:

```
"REPORT: Ameen (22) | Course: CS | Score: 85 | Grade: B – Very Good | Status: PASSED"
```

Test with:

- `{ name: "Ameen", age: 22, score: 85, course: "CS" }`
- `{ name: "Jane", age: 19, score: 45 }` (no course)
- `{ name: "Mark", age: 20 }` (no score, no course)
- `{}` (empty object)

// Write your generateReport function and tests below:

Q15. BONUS CHALLENGE: processStudents(studentsList)

5 Bonus Marks

Write a function `processStudents` that takes an array of student objects, iterates using `.forEach()`, calls `generateReport()` for each, and finally prints:

```
"Total students: X | Passed: Y | Failed: Z"
```

```
const testStudents = [
  { name: "Ameen", age: 22, score: 85, course: "CS" },
  { name: "Jane", age: 19, score: 45 },
  { name: "Mark", age: 20, score: 72, course: "Math" },
  { name: "Ada", age: 21, score: 38, course: "Physics" },
  { name: "Kemi", age: 18, score: 90, course: "Biology" },
];
```

// Write your bonus function below:

Section	Topics Covered	Maximum Marks	Student Score
Section A	Variables (let / const), Data Types, typeof Operator	10	
Section B	Type Coercion Traps, Strict Equality (=== vs ==)	10	
Section C	Conditionals (if/else), Ternary Operator, Logical AND (&&)	15	
Section D	Optional Chaining (?.), Nullish Coalescing (??)	10	
Section E	Functions (Declaration, Expression, Arrow Syntax, Defaults)	30	
Section F	Capstone Integration (Report Generator & Edge Cases)	25	
BONUS	Array Iteration with forEach & Counters	5	
TOTAL SCORE		105	

JavaScript Fundamentals Drilling Assignment • Produced for Classroom Instruction • Good Luck!